TP5-6

Test avec JUnit

TP 5 : Mise en place de JUnit

5.1 Objectif

L'objectif de ce TP est de tester votre package `fr.univrennes.istic.l2gen.geometrie` à l'aide de JUnit.

5.2 Installation

Vérifier que JUnit est correctement installé dans VS Code. Sinon, l'installer.

5.3 Création d'un dossier de test (un seul membre de l'équipe)

A la racine de votre projet, c'est-à-dire au même niveau que votre dossier `src`, créer un dossier `test`. Dans celui-ci, créer un dossier `fr/univrennes/istic/l2gen/geometrie` afin de permettre d'ajouter vos classes de test au package `fr.univrennes.istic.l2gen.geometrie`. Votre hiérarchie de packages de tests devra toujours suivre une forme parallèle à la hiérarchie du dossier `src`.

5.4 Partage du dossier test

La personne ayant crée le dossier test ci-dessus l'enregistre dans git (add, commit puis push) comme vu au TP précédent. Les autres font un **pull** sur le projet et vérifient qu'ils ont bien récupéré ce dossier test et son contenu.

5.5 Écriture des tests unitaires

Question 1 *Répartissez-vous le travail pour que pour chacune des classes de votre package* `fr.univrennes.istic.l2gen.geometrie`*, il soit créé dans* `test/fr/univrennes/istic/l2gen/geometrie` *une classe de test JUnit.*

Pour accélérer la saisie, vous pouvez demander à VS Code de générer le squelette d'une classe de test pour vous. Pour cela, dans l'éditeur d'une classe (par exemple Cercle) faites : click droit>source Action>generate tests. Déplacer alors la classe obtenue (CercleTest) dans le dossier `fr/univrennes/istic/l2gen/geometrie`.

Par exemple pour la classe Cercle on devra avoir une classe CercleTest ressemblant à :

```
package fr.univrennes.istic.l2gen.geometrie;
import static org.junit.Assert.*;
import org.junit.Test;

public class CercleTest {
        @Test
```

```
        public void testDescription() {
        //...
        }

        @Test
        public void testDupliquer() {
        //...
        }

        @Test
        public void testEnSVG() {
        //...
        }
 //...
}
```

Ecrire le contenu de ces tests en suivant les instructions vues en cours, en particulier en se servant des différentes formes d'`assert` disponibles dans `org.junit.Assert.*`. Toutes les méthodes de chaque classe doivent être testées ainsi.

Pensez notamment à utiliser les préludes de test pour factoriser les parties communes, par exemple pour Cercle :

```
        Cercle c;
        @Before
        public void setUp() {
        c = new Cercle(100,100,50);
        }
  }
```

5.6 Exécution des tests unitaires

Lancez vos tests à l'aide de

Si un test montre une erreur, celle-ci peut venir soit du test soit du code de la classe testée. Corrigez toutes les erreurs puis enregistrez vos modifications dans gitlab. Corrigez d'éventuels conflits. Puis mettez à jour avec git pull.

Important : On ne supprime/commente jamais un test quand celui-ci passe !

JUnit est un outil de tests unitaires **automatisés** :

— Chaque test est écrit une seule fois, mais il peut-être relancé à volonté (en particulier après toute modification)

— Les tests sont sauvegardés/versionnés sur le GitLab du projet

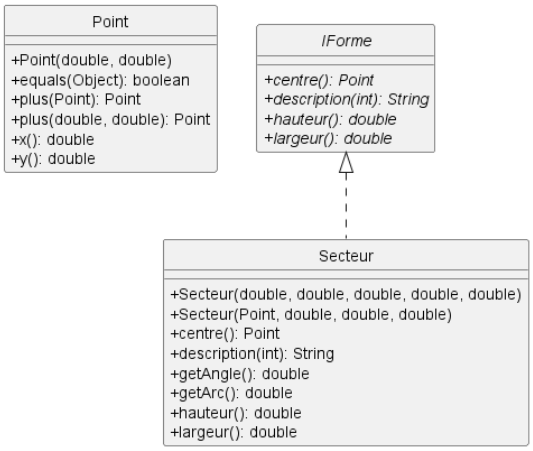
Un test est plus précieux (et pérenne) qu'une ligne de code, car il va dans le futur servir de test de non-régression. Un test de non-régression vérifie en effet que l'ajout de nouvelles fonctionnalités dans un programme ne perturbe pas les fonctions initiales.

TP 6 : Séparation codage et test

Vous allez développer en parallèle (1) le code et (2) les tests de la classe Secteur comme ci-dessous.

Un Secteur (de cercle) est caractérisé par le centre et le rayon du cercle, ainsi qu'une partie visible constituée d'angle de départ et un arc en degrés.

La classe Secteur du package `fr.univrennes.istic.l2gen.geometrie` implémente l'interface IForme comme décrit ci-contre.



Question 1 *Séparer votre équipe en 2 sous-équipes : les codeurs et les testeurs. Les testeurs écrivent les tests TestSecteur pendant que les codeurs implémentent la classe Secteur. Synchronisez-vous au travers de Git assurez-vous que tous les tests passent bien. Répartissez-vous le travail en faisant en sorte que chaque binôme testeur/implémenteur réponde à une des questions ci-dessous qui ont simplement pour objectif de vous aider à implémenter et tester les comportements attendus des principales méthodes.*

Question 2 *Implanter la méthode description()*

L'exécution du programme suivant :

```
IForme f = new Secteur(256, 256, 128, 0, 60);
System.out.println(f.description(1));
```

doit donc afficher exactement le texte :

```
 Secteur centre=256,256 r=128.0 Angle=0.0 Arc=60.0
```

Question 3 *Implanter la méthode redimensionner(). Comme pour les autres classes du package* `geometrie`*, on vérifiera que lorsqu'on construit un arbre avec la méthode présentée au TP1 :*

```
Groupe arbre = arbre(new Secteur(256, 256, 128, 0, 60));
System.out.println(arbre.description(0));
```

L'exécution de ce programme doit afficher le texte suivant, reflétant la structure d'arbre grâce à l'indentation :

```
Groupe
  Secteur centre=256,256 r=128.0 Angle=0.0 Arc=60.0
  Secteur centre=256,256 r=64.0 Angle=0.0 Arc=60.0
  Groupe
    Secteur centre=256,256 r=32.0 Angle=0.0 Arc=60.0
    Secteur centre=256,256 r=16.0 Angle=0.0 Arc=60.0
```

Question 4 *Implanter la méthode enSVG()*

L'exécution du programme :

```
IForme f = new Secteur(256, 256, 128, 0, 60);
System.out.println(f.enSVG()));
```

doit par exemple afficher le texte :

```
<path d="M256␣256␣l0,-128␣A␣128.0␣128.0␣0␣0␣1␣367,192␣Z"
    fill="white" stroke="black"/>
```

Appeler `arbre.enSVG()` sur l'arbre défini à la question précédente pour obtenir :

```
<g>
<path d="M256␣256␣l0,-128␣A␣128.0␣128.0␣0␣0␣1␣367,192␣Z"
    fill="white" stroke="black"/>

<path d="M256␣256␣l0,-64␣A␣64.0␣64.0␣0␣0␣1␣311,224␣Z"
    fill="white" stroke="black"/>

<g>
<path d="M256␣256␣l0,-32␣A␣32.0␣32.0␣0␣0␣1␣284,240␣Z"
    fill="white" stroke="black"/>

<path d="M256␣256␣l0,-16␣A␣16.0␣16.0␣0␣0␣1␣270,248␣Z"
    fill="white" stroke="black"/>

</g>
</g>
```

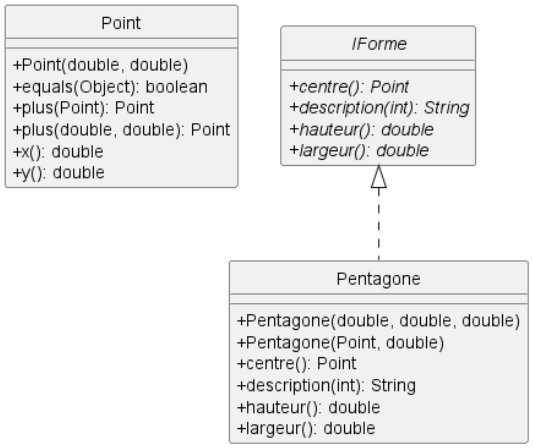
Attention : A l'issu de ce TP, votre encadrant vous fera un premier retour sur la complétude et la qualité de votre code tel que trouvé dans votre dépôt GitLab. Il est donc impératif de respecter le nommage de tous les éléments d'interface.

Question 5 *Obligatoire pour le parcours Défi, facultative pour les autres.*

Faites de même pour la classe Pentagone.

Un Pentagone (régulier) est caractérisé par un centre et une hauteur. Ses 5 cotés ont donc la même longueur.

La classe Pentagone du package `fr.univrennes.istic.l2gen.geometrie` implémente l'interface IForme comme décrit ci-contre.



Question 6 *Obligatoire pour le parcours Défi, facultative pour les autres.*

Généraliser à tout polygone régulier (Hexagone, Octogone...)